



# Arquitetura de Software e Padrões

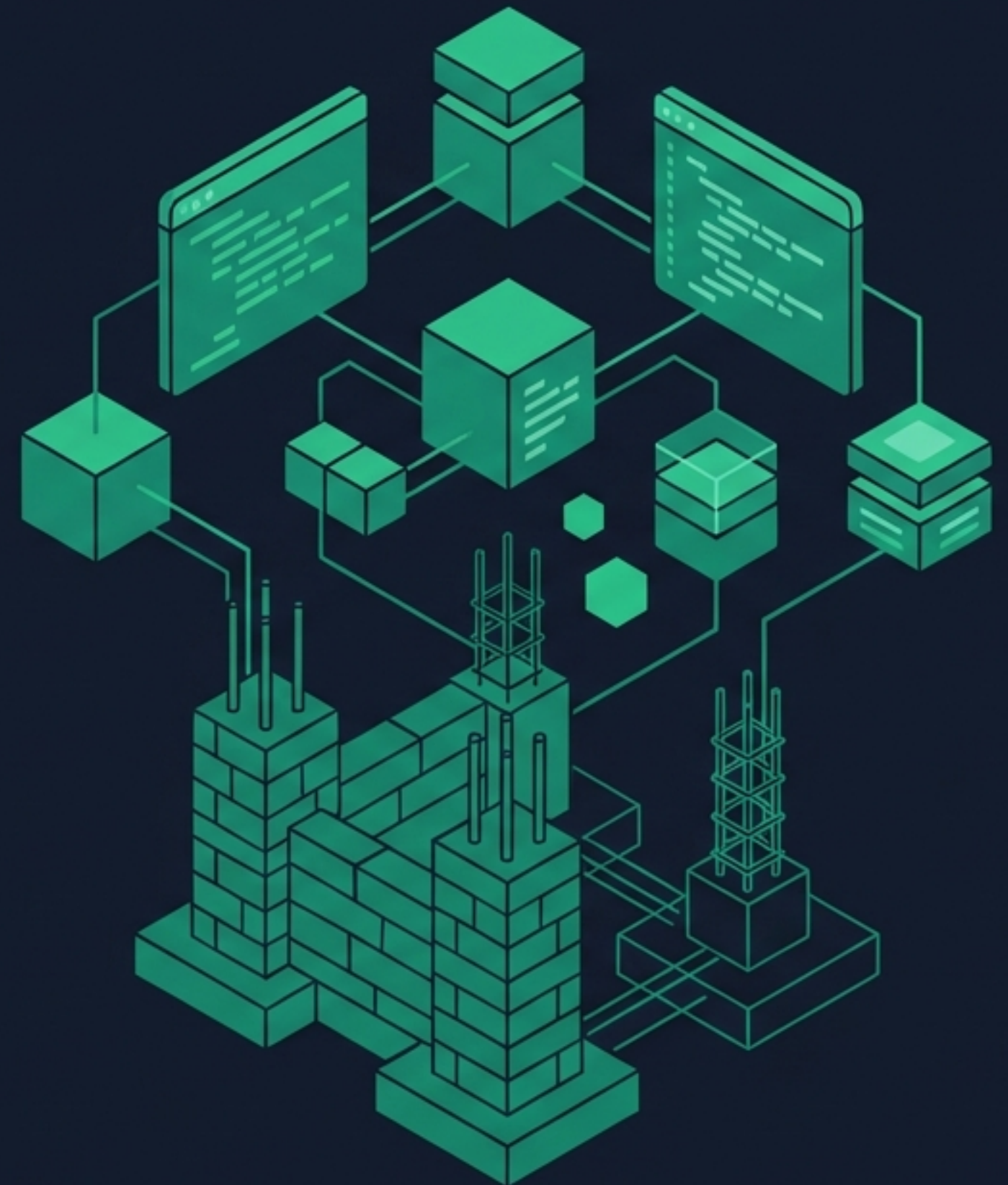
A Estrutura e a Construção

Grupo 3 | Curso Técnico de Informática



# Muito Além da Linha de Código

- **O Esqueleto do Sistema:** Arquitetura não é apenas escrever código limpo ou usar princípios SOLID. É a visão macro.
- **Decisões Caras:** São as escolhas estruturais que são tecnicamente difíceis e custosas de mudar após a implementação.
- **A Fundação (Blueprint):** Define como os elementos interagem para garantir escala, segurança e resiliência.





# O Que Forma a Estrutura?

## Os três pilares de Perry & Wolf



- **Processamento:** A lógica computacional. Os componentes que transformam os dados.



- **Dados:** A informação que é armazenada, processada e transmitida pelo sistema.



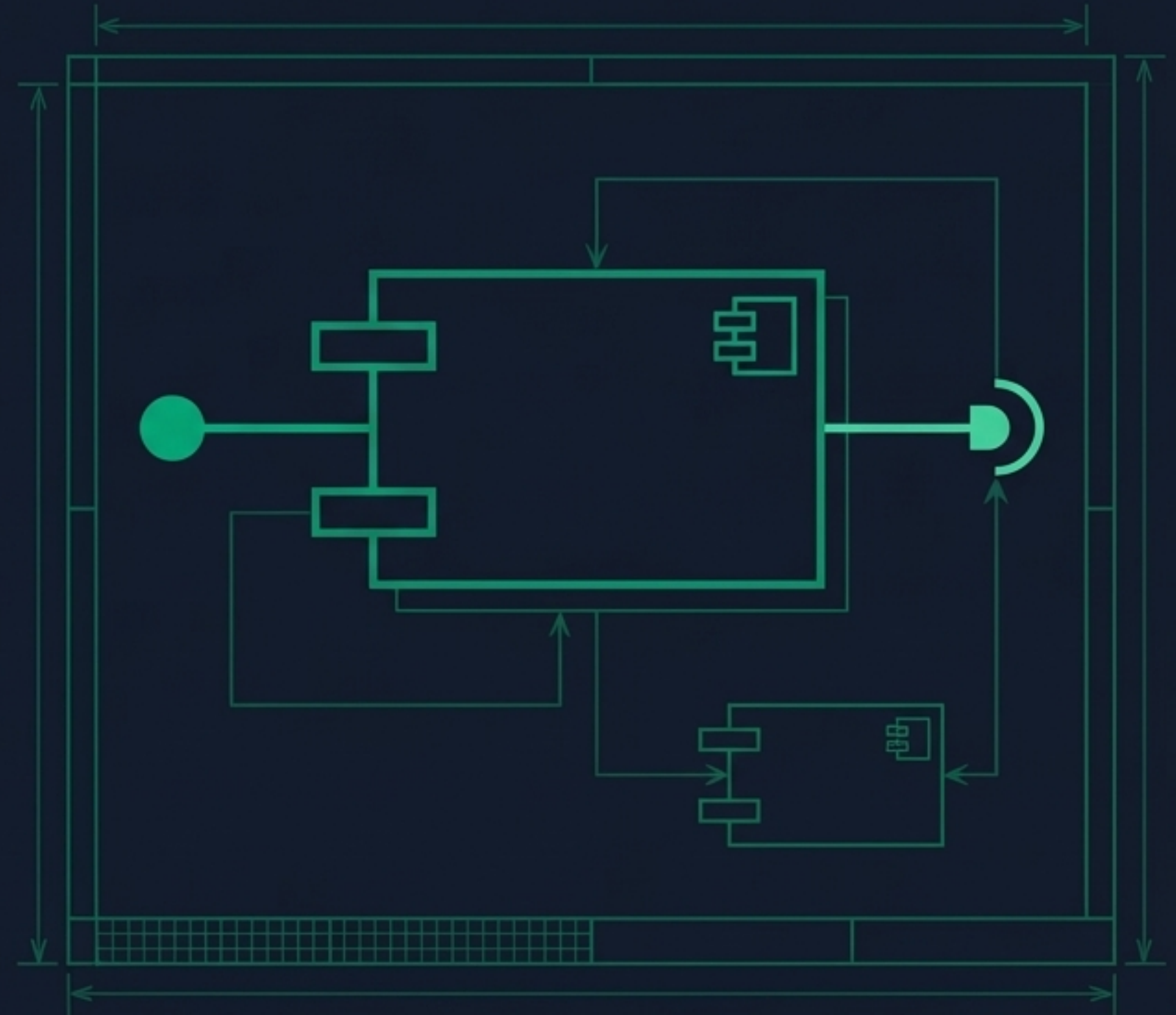
- **Conectores (A "Cola"):** Os mecanismos e protocolos que unem o processamento e os dados (APIs, comunicação, sincronização).



# Diagramas Físicos: A Visão Lógica

## Diagrama de Componentes (O “Que” e o “Como”)

- **Foco:** Organização lógica e modularidade do software.
- **Componentes:** Unidades de funcionalidade independentes e substituíveis (ex: uma biblioteca .dll ou .jar).
- **Interfaces (Lollipop):** O contrato que o componente oferece para os outros.
- **Conectores:** Linhas de dependência mostrando como os módulos conversam entre si.

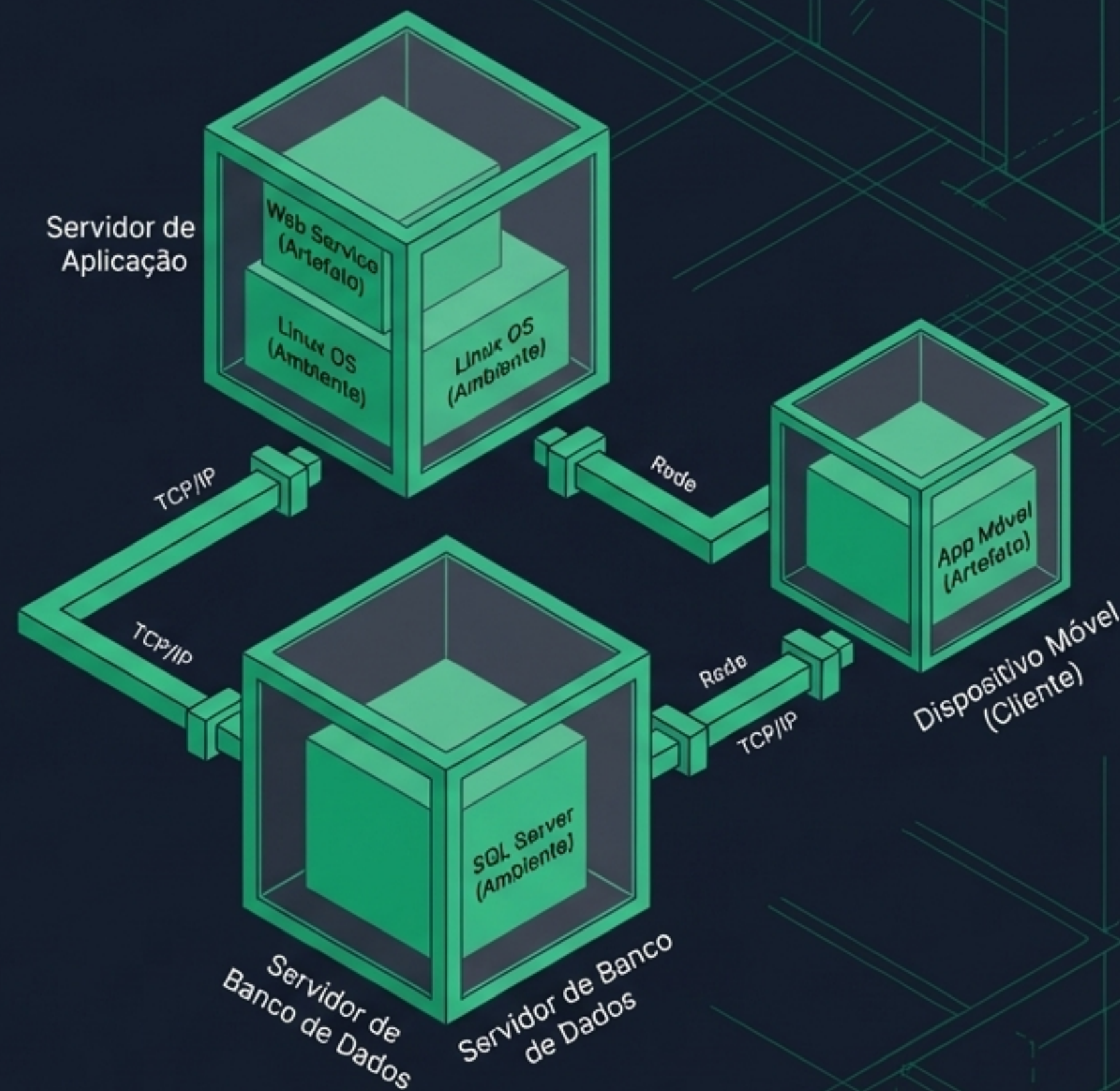




# Diagramas Físicos: A Visão Física

## Diagrama de Implantação (O "Onde")

- **Foco:** A arquitetura de hardware e a distribuição na rede. Planejamento de infraestrutura.
- **Nós (Nodes):** Recursos computacionais reais físicos ou virtuais (Servidores, PCs, Smartphones).
- **Ambientes de Execução:** Sistemas operacionais ou bancos de dados (ex: Linux, SQL Server).
- **Artefatos:** O software compilado (arquivos físicos) implantado dentro dos nós.

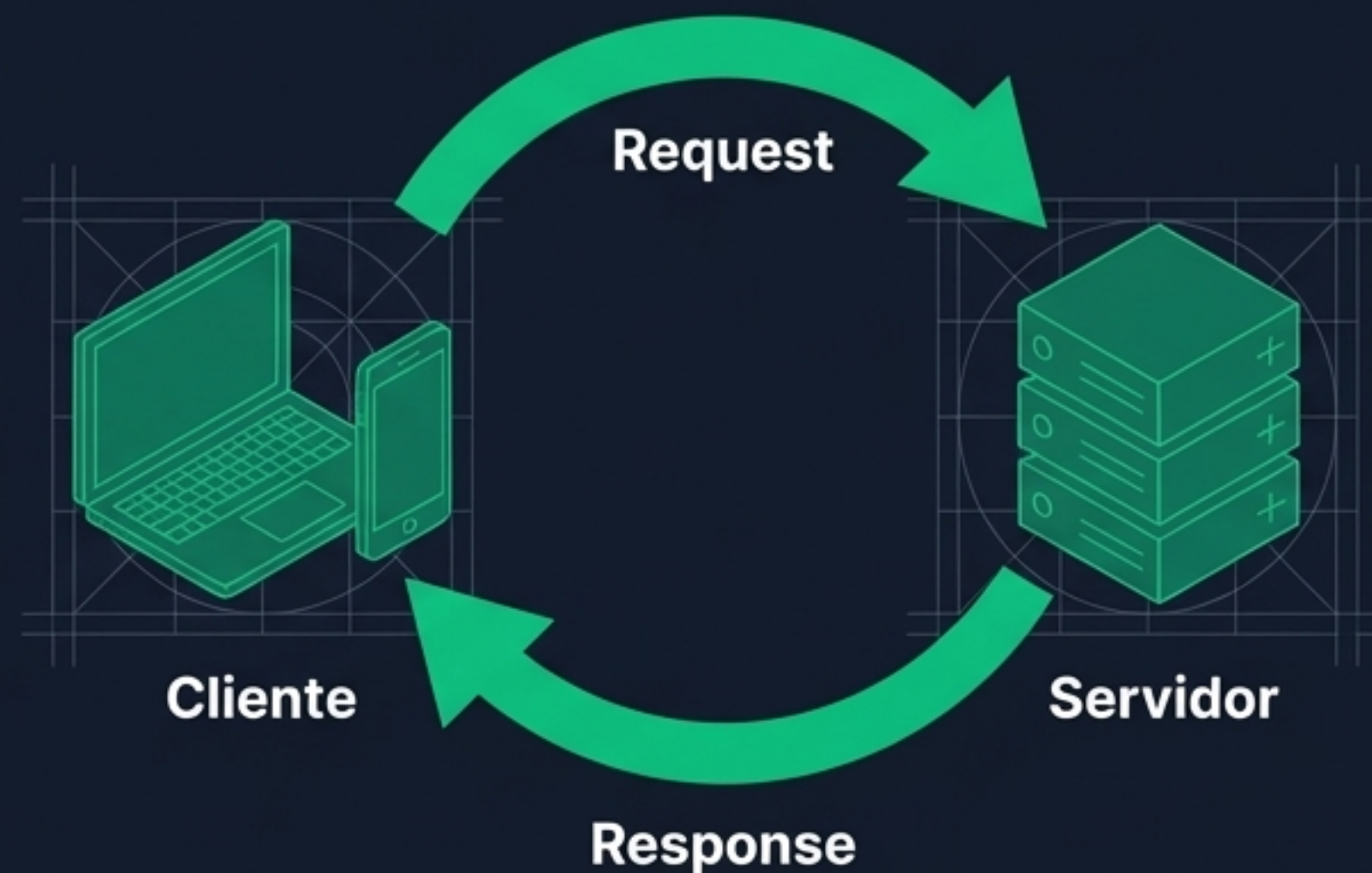




# Grandes Padrões: Cliente-Servidor

## A base da internet moderna

- **O Padrão:** Separação clara entre quem solicita recursos e quem os processa.
- **Frontend (O Cliente):** Interface do usuário. Foco em renderização visual, experiência e envio de requisições.
- **Backend (O Servidor):** Gerenciamento centralizado. Foco em lógica pesada de negócios, acesso ao banco de dados e segurança.
- **Comunicação:** Ciclo estruturado de Request (pedido) e Response (resposta).





# Grandes Padrões: O Monolito

## O "Tudão" integrado

- **O Conceito:** Toda a aplicação (UI, regras, acesso a dados) compilada e implantada como uma única unidade indivisível.
- **Vantagens:**
  - Baixo custo inicial e simplicidade.
  - Ideal para startups criando um MVP (Produto Mínimo Viável) rápido.
- **Desvantagens:**
  - Difícil escalar. Se um módulo falha, o sistema inteiro pode cair.
  - Qualquer mudança exige a reimplantação de todo o software.





# Grandes Padrões: Microserviços

## O "Dividido" e descentralizado

- **O Conceito:** A aplicação é dividida em serviços pequenos, autônomos e focados em negócios específicos, comunicando-se por APIs.
- **Vantagens:**
  - Escala granular (escale apenas o que precisa, ex: só o carrinho de compras).
  - Isolamento de falhas (se um cai, o resto funciona).
- **Desvantagens:**
  - Alta complexidade operacional.
  - Exige forte infraestrutura de DevOps e orquestração.

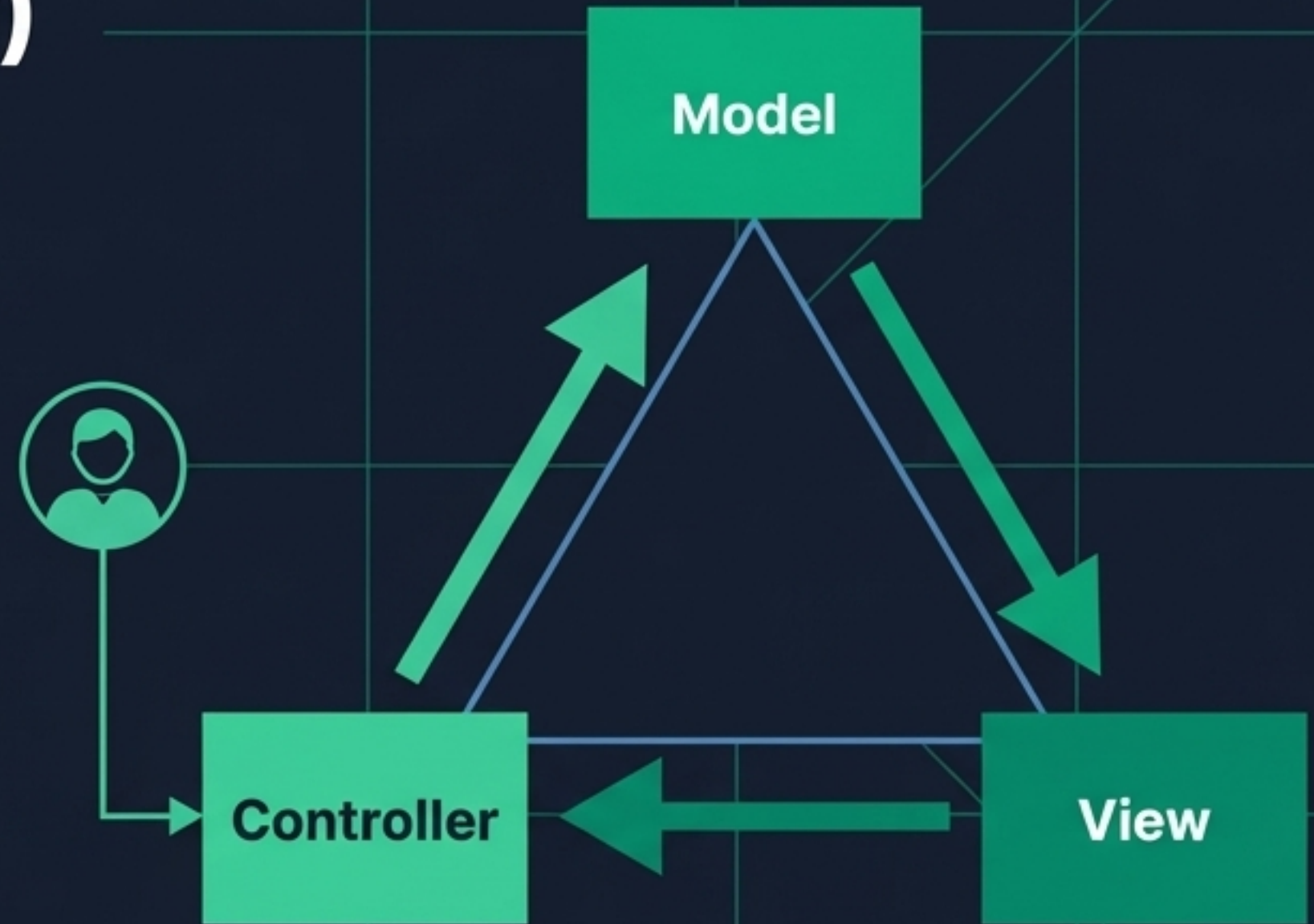




# Grandes Padrões: MVC (Model-View-Controller)

## Organizando interfaces dinâmicas

- **Separação de Preocupações:** Divide a aplicação em três responsabilidades principais.
- **Model (Modelo):** O cérebro. Regras de negócio, manipulação de dados e estado. Não sabe como a tela é desenhada.
- **View (Visão):** O rosto. O HTML/Interface apresentada ao usuário.
- **Controller (Controlador):** O maestro. Recebe a entrada do usuário, aciona o Modelo e escolhe qual Visão mostrar como resposta.





# Conclusão: A Primeira Lei da Arquitetura



- **"Tudo é um Trade-off"** (Uma troca)
- Não existe arquitetura perfeita ou "a melhor". Existe a escolha certa para o problema de negócio atual.
  - Compreender a teoria, dominar os diagramas (Componentes e Implantação) e conhecer os padrões (Monolito, Microserviços, MVC) é o que diferencia o programador de um verdadeiro Arquiteto de Software.